

System testing (test cases and results)

This document lists all the test results from system testing that have been received using different testing environments. Tests have been done for both the web and mobile versions of the software.

1. Mobile test cases

TEST CASE 1: SENDING AND RECEIVING HELP REQUEST

Test purposes

1. This test case acts as a general system integration test to see if everything works as it should.
2. Ensures all components of the system are working correctly (database, local server, React client, Flutter app)
3. Ensures help requests can be sent to the local backend server.
4. Ensures that the backend server sends the new help request to all local users in the area.

Test procedure

Let's test this feature step by step:

1. On both emulators, make sure that all 6 different views (accessible from the burger menu - nav bar) are working and they are showing images
2. From the "requester" device, press the "Call for help" button, select any of the 3 help categories, and create a new help request.
3. The requester's device should enter the "Helping" view. It should show that there are currently 0 helpers in this session (since we haven't accepted the request on our helper device yet.)
4. On the "helper" device, a notification should be showing that "Someone is calling for help" in a new view.
5. We can continue to test Accepting & Helping a request from here.

Acceptance criteria

Acceptance criteria & possible errors + their causes from each of the testing steps:

1. All 6 views are visible, without errors. If something unusual shows up, it could potentially be because of:
2. The local backend server hasn't started.
3. The local React client is down → the Flutter app cannot fetch the WebView from the React client.

4. Users should be able to create new help requests. If not, here are some of the possible causes:
5. Errors with the local backend server. Check the server's log!
6. The requester should be able to see the "Helping" view for their newly created help-requesting session. If not, possible causes for errors are:
7. Possible errors with the local backend server. Check the server's log! (this can happen quite often)
8. The location of the "helper" device is not set to the same proximity (< 3km) from the requester's device. If this happens, the view will show "No active users around your location".
9. The "helper" device should be able to receive the help request notification. If not, possible causes are:
10. Possible errors with the local backend server. Check the server's log! (this can happen quite often)
11. The location of the "helper" device is not set to the same proximity (< 3km) from the requester's device. If this happens, the view will show "No active users around your location".

TEST CASE 2: ACCEPTING AND HELPING A REQUEST

Test purpose

1. This test is an integration test between the mobile app and the server. It tests if help requests are being processed correctly.
2. Ensures help requests are saved in the database as sessions.
3. Ensures server can communicate between requester and helpers.
4. Ensures termination criteria of the help sessions are met.

Test procedure

This test acts as a continuation after the Sending new help request test.

1. Let's accept the help request using the confirm button on the helper's device.
2. Both devices should now be in the "Helping" view, showing a map with dots representing the requester and the helper's location. It is possible to continue to the Help request instant messaging test after this step.
3. On both devices, try panning, spinning the map around, and interacting with it to see if it is usable. We can also try to change our location (in 1 of the 2 devices). Don't change to somewhere too far (the distance between 2 emulators should be <3km).
4. Now, let's test the termination of a help session. There can be 2 options. All need to be tested.
5. The requester can press the "Cancel help request" button on the screen to end the help session.
6. Either party moves too far (>3km) away from each other. With either of the emulators, change the location of that device to somewhere far away (e.g., Hervanta campus to city center, for example)

Acceptance criteria

1. The helper shall be able to accept the help request sent by the requester.
2. Both devices shall show the “Helping” screen. Both devices shall display a map, with 1 dot representing the requester’s location and another dot representing the helper's current location.
3. Both users shall interact with the map. They can pan, zoom, and rotate the map with ease. The dots representing the requester & helper should accurately reflect their real-life location (based on the 2 devices' GPS data).
4. The helping session shall be terminated when either condition listed in the test procedure is fulfilled. For the requester, their device shall go back to the app's home screen. For the helper, their device should show a message “The help request has ended”, and then go back to the app's home screen.

TEST CASE 3: HELP REQUEST INSTANT MESSAGING TEST

Test purposes

1. This test case is an integration test between the Flutter app and the Firebase database. The Flutter app shall communicate with Firebase effectively to ensure the reliability of the rescue’s chat messaging service.
2. Ensures all components of the rescue element’s real-time messaging are working correctly (Firebase, Flutter app)
3. Ensures requester and helpers can communicate anonymously and effectively during a help-requesting session

Test procedure

This test can start after Step 2 in the Accepting & Helping a request test: the requester and helper devices are in the “Helping” view. There should be 2 parts of this test: testing the sending & receiving functionality of the chat room, and deleting of stale chat data when the rescue request is ended.

Sending & Receiving messages

1. The requester device presses on the text message icon on the top right of the view. This shall open the chat dialog.
2. In the chat dialog: there should be 2 person avatars on the top header of this chat dialog. One is the requester, with a red color and a label “Requester” underneath. The other one is the helper, with a unique color (different than red), and a label “Helper 1” underneath.
3. The requester can enter any message into the text field of the dialog. Press the send icon on the right of the text field, or the enter key on the device’s on-screen keyboard to send the text message.

4. After the message is sent, it should appear at a text bubble in the chat dialog, with the same red color as the requester's avatar in the header.
5. On the helper device, a new notification shall pop up, saying that there is a new unread chat message. The icon of the chat message shall now have a red dot, further indicating that there are unread messages.
6. The helper device presses on the chat message icon to open the same chat dialog and read the message. They can also see the same message from the requester device in a red bubble.
7. Helper device can close the dialog. They shall see the red dot notification icon on the chat icon from earlier (step 5) is gone now, indicating that they have read all of the messages.
8. The helper device can open the dialog again. They can send another message to the chat room, following the same procedure described in step 3 and send it.
9. After the new message is sent, the helper device should see their new message appear in the chat room, in a chat bubble with the same color as their avatar on the top header of the chat dialog (described in step 2).
10. This time, the helper device should be the one to receive the unread message notification. Check the description in step 5.
11. Helper device can open the chat dialog and view the message, then close it, to see that the indicator is gone. As described in step 6 and 7.
12. The helper and requester device can repeat step 3 → 11 multiple times, sending messages back and forth to see if everything works fine.

Deleting of chat data when the request ends

The chat room should automatically delete its chat data when the current chat request ends → No data persistence, and each help request should have only 1 chat room instance. This test can continue from step 11 of the last test.

1. After testing the chat messaging function, the requester device can end the help session using the "Stop calling for help" button on the top of the helping view.
2. Let's check if the chat data is deleted correctly. After returning to the home screen, the requester can create a new help request. Follow the steps described in Sending & Receiving help requests.
3. The helper device should now accept this new help request created.
4. The requester device and the helper device shall now enter a new help session, with the same view and functionalities described in Accepting & Helping a request.
5. The requester can click on the same chat icon described in step 1 of the last test. They shall see a brand new, blank chat room opens.

Acceptance criteria

All of the helpers and requester of a help session can open, send, and receive messages in the instant chat room of a help session.

The app send a new message notification, with a red dot indicator on the chat icon, when any of the other parties send a new chat message.

The chat messages should have the same bubble color as their sender's avatar color. The avatars are located on the top header of the chat room dialog.

The chat room should automatically delete all of its chat data when the help session ends.

2. Web test cases

Web	
1	Changes language back and forth
2	Opens lumisovellus.fi with phone and changes language
3	Opens weather tab
4	Opens definitions tab
5	View Skiing area only
6	Check sensors information
7	Hide and show sensors
8	Zoom in and out of map
9	Add observation
10	Navigates to add new snow estimation
11	Adds new snow estimation and checks that it is now in feedback tab

3. Result

2.1 Mobile test

Test information

Project	Snowledge
Date	15.12.2023
Tester	Khoa Nguyen
Test phase	System testing
Version	2.1.0
Test environment	Android Mobile phone
Results	PASS
Accepted	Yes

Test result

Test case	Result	Details
Sending and receiving help request	Passed	
Accepting and helping a request	Passed	

Help request instance message testing	Passed	
---------------------------------------	--------	--

2.2 Web test

Test information

Project	Snowledge
Date	15.12.2023
Tester	Chinh Pham
Test phase	System testing
Version	2.1.0
Test environment	Microsoft Edge
Results	PASS
Accepted	Yes

Test result

Test case	Result	Details
1	Passed	
2	Passed	
3	Passed	
4	Passed	
5	Passed	
6	Passed	
7	Passed	
8	Passed	
9	Passed	
10	Passed	
11	Passed	